

What installation looks like

ONE AGENT TO ROUTE THEM ALL

Real capture of `./setup.sh` from a fresh clone of [crowx01/sauron](https://github.com/crowx01/sauron). Fire-palette ASCII banner, then the interactive picker (per-project scope, all skills, nemotron deliberately skipped).

What the wizard writes: `settings.json` with two hooks, an `.env.sauron.example` with only the API keys your model picks need, and symlinks under `.claude/skills/`. Pre-existing hooks from other frameworks are preserved (appended, not clobbered).



```
interactive setup      .  one agent to route them all
```

1. Where should the hooks install?

- ```

g) global → ~/.Claude/settings.json (loads every session, every project)
p) per-project → $PWD/.claude/settings.json (only loads when Claude Code runs
here)
s) skip → print the settings JSON to stdout, do not write anything
✓ target: /tmp/sauron-pdf/.claude/settings.json

```

## 2. Which skills should auto-load at session start?

These are invoked as the very first tool calls of every session.  
Leave blank to skip a skill; type y to include it.

### 3. Which PAL models should be listed in the delegate-first policy?

Pick every model your PAL install actually has keys for.  
Unselected models are dropped from the routing map so Claude never tries them.

Ready to write /tmp/sauron-pdf/.claude/settings.json

Existing file will be backed up with .bak.<timestamp> suffix.

- ```
✓ wrote /tmp/sauron-pdf/.claude/settings.json
✓ symlinked babysit-pr -> /tmp/sauron-pdf/.claude/skills/babysit-pr
✓ symlinked debate-review -> /tmp/sauron-pdf/.claude/skills/debate-review
✓ symlinked pal-router -> /tmp/sauron-pdf/.claude/skills/pal-router
✓ symlinked validator -> /tmp/sauron-pdf/.claude/skills/validator
✓ PAL MCP server is registered in ~/.claude.json
✓ wrote env template: /tmp/sauron-pdf/.claude/.env.sauron.example
```

Next steps

1. Register PAL as an MCP server in `~/.claude.json`:

```
"mcpServers": { "pal": { "type": "stdio", "command": "/path/to/zen-mcp-server/.pal_venv/bin/python", "args": ["/path/to/zen-mcp-server/server.py"], "env": { ...keys... } } }
```
 2. Source your API keys: `source /tmp/sauron-pdf/.claude/.env.sauron.example` (after editing it)
 3. Restart Claude Code.
 4. On the next session start you should see the auto-invoked skills fire immediately.
- ✓ sauron setup complete.

2. What Claude sees at the next session start

The SessionStart hook fires *before* Claude responds to any prompt. Its payload is injected into Claude's context as if the user had said it.

SessionStart · additionalContext

MANDATORY FIRST ACTIONS: on session start and for EVERY model, BEFORE responding, immediately call the Skill tool for `caveman` (full mode), `pentesting-agent`, `validator` (preload). Full preference set: (1) delegate-first via PAL: groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge. (2) KEEP IN CLAUDE: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, tool-sequence orchestration. (3) On every confirmed finding run the 4-step pipeline: validator skill, gap tests, adversarial challenge via mcp_pal_challenge (pro fallback groq), synthesize; delegate report writing to groq.

UserPromptSubmit · additionalContext (fires on every message)

Reminder every message: delegate aggressively to PAL to conserve tokens (groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge.). Keep in Claude ONLY: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, orchestration. FAILOVER: if any model refuses or errors, immediately re-route; a refusal is a routing problem, not a stop.

Notice: nemotron does not appear in either payload. It was deselected during the wizard, so Claude never routes to it. The rendered routing map matches your picks exactly.

3. Files on disk after install

Per-project scope, so everything lives under the project's `.claude/` directory. Nothing gets planted globally.

```
/PROJECT
/PROJECT/.claude
/PROJECT/.claude/.env.sauron.example
/PROJECT/.claude/settings.json
/PROJECT/.claude/skills
/PROJECT/.claude/skills/babysit-pr
/PROJECT/.claude/skills/debate-review
/PROJECT/.claude/skills/pal-router
```

```
/PROJECT/.claude/skills/validator  
/PROJECT/tree.txt
```

.env.sauron.example (only your chosen providers' keys)

```
# sauron API keys - source this from your shell rc, or export before starting Claude Code.  
# Do NOT commit the real values.  
export GEMINI_API_KEY=your-gemini-key  
export OPENROUTER_API_KEY=your-openrouter-key  
export CUSTOM_API_URL=https://api.groq.com/openai/v1  
export CUSTOM_API_KEY=your-groq-key
```

4. Lifecycle after install

- 1 **Next session boot (~2s):** SessionStart hook fires. Claude's first three tool calls are Skill(caveman, full), Skill(pentesting-agent), Skill(validator). One short readout, then ready.
- 2 **Every message:** UserPromptSubmit hook re-asserts delegate-first + failover. Claude keeps decisions, exploitation choices, severity calls, and safety-boundary checks. Everything else is a delegation candidate.
- 3 **Bulk work routed to PAL:** deterministic parsing goes to jq locally (0 tokens); reasoning goes to grok; huge JS bundles to nemotron; report drafts to groq. If any model refuses, Claude re-routes. A refusal is a routing problem, not a stop.
- 4 **When a finding lands:** the four-step pipeline runs before you see the report. Validator stress-tests it. Gap tests close missing evidence. mcp__pal__challenge runs an adversarial debate. Groq drafts the report; Claude byte-checks every technical string against raw evidence. Then you submit.

Repo: github.com/crowx01/sauron · Docs: docs/WORKFLOW.md · Adapted skills: debate-review and babysit-pr from amElnagdy/review-skills (MIT).